

Projet d'évaluation - Wordle

Contexte

Pour valider ce module sur les tests unitaires, vous allez développer le jeu **Wordle** en mettant l'accent sur la **modélisation du domaine métier** et sa **couverture par des tests**.

Vous êtes libres d'utiliser le langage et le framework de test de votre choix (TypeScript/Vitest, Java/JUnit, C#/xUnit, Python/pytest, etc.). Les exemples de ce sujet sont donnés en TypeScript à titre indicatif.

L'objectif est de démontrer votre capacité à :

1. **Modéliser** un problème métier avec des types expressifs
2. **Isoler** le code métier de toute dépendance technique (affichage, fichiers, réseau)
3. **Tester** le comportement de votre domaine de manière exhaustive et lisible
4. **Livrer** une application fonctionnelle (CLI ou interface graphique)

Règles du jeu "Wordle"

- Le système choisit un mot secret de **5 lettres**.
- Le joueur a **6 tentatives** pour deviner ce mot.
- Après chaque tentative, chaque lettre soumise reçoit une évaluation (feedback) :
 -  **CORRECT** : La lettre est présente dans le mot secret et se trouve à la bonne position.
 -  **MISPLACED** : La lettre est présente dans le mot secret, mais à la mauvaise position.
 -  **ABSENT** : La lettre n'est pas présente dans le mot secret.

⚠ Règle spécifique (Lettres Multiples) :

Si une lettre apparaît plusieurs fois dans la proposition du joueur, mais moins de fois dans le mot secret, elle ne doit être marquée "CORRECT" ou "MISPLACED" qu'à concurrence de son nombre d'occurrences dans le mot secret. Les occurrences surnuméraires doivent être marquées "ABSENT".

Exemple : Mot secret = LIVRE , Proposition = RAMER

- *R (pos 1) →  MISPLACED (R existe dans le secret, mais pas à cette position)*
- *A (pos 2) →  ABSENT*
- *M (pos 3) →  ABSENT*

- *E* (pos 4) →  MISPLACED
- *R* (pos 5) →  ABSENT ← Il n'y a qu'un seul *R* dans "LIVRE", déjà consommé par le premier *R*

Comment aborder le sujet

Il n'y a pas de code de démarrage fourni. Ce projet vous demande de **partir de zéro** et de réfléchir à la conception avant d'écrire du code. Voici une démarche recommandée.

Étape 1 — Identifier les concepts métier

Avant d'écrire la moindre ligne de code, posez-vous la question : **quels sont les éléments fondamentaux d'un Wordle ?**

On a par exemple le concept de **lettre**, de **mot** (et plus précisément d'un mot valide de 5 lettres), le concept de **dictionnaire** qui contient les mots valides, etc. Quels autres concepts sont nécessaires pour représenter une partie complète ?

Pour vous lancer, voici deux types de base que vous pouvez reprendre ou adapter dans votre langage :

```
// Un mot valide garanti de 5 lettres
type Word = string & { readonly _brand: "Word" };

// Le feedback pour une lettre donnée
type LetterFeedback = "CORRECT" | "MISPLACED" | "ABSENT";
```

Le reste de la modélisation (résultat d'une tentative, état de la partie, etc.) est à votre charge.

Étape 2 — Modéliser les comportements

Une fois vos concepts identifiés, demandez-vous : **que se passe-t-il quand... ?**

- Le joueur soumet un mot : qu'est-ce qui est retourné ?
- Le joueur trouve le bon mot : que devient la partie ?
- Le joueur épuise ses 6 tentatives : que se passe-t-il ?
- Le joueur soumet un mot qui n'existe pas dans le dictionnaire ?
- Le joueur essaie de jouer alors que la partie est déjà terminée ?

Chaque question est un **comportement** à implémenter et à tester.

Pensez aussi à modéliser les **erreurs** : que retourne votre domaine quand un mot est invalide ? Quand la partie est déjà terminée ? Les erreurs sont des concepts métier à part entière — elles méritent des types dédiés plutôt qu'un simple `throw new Error("...")`.

Par exemple, en TypeScript, vous pouvez définir une erreur custom en étendant la classe `Error` :

```
class InvalidWordError extends Error {
  constructor(word: string) {
    super(`"${word}" is not a valid 5-letter word`);
    this.name = "InvalidWordError";
  }
}
```

Cela vous permet ensuite de tester précisément le type d'erreur levée dans vos tests (`expect(...).toThrow(InvalidWordError)`).

Étape 3 — Écrire les tests, implémenter, itérer

Je vous recommande l'approche **Test First** : écrivez le test qui décrit le comportement attendu, puis écrivez le code qui le fait passer. Ce n'est pas une obligation, mais c'est une démarche efficace pour avancer méthodiquement.

Structurez vos tests en **Given / When / Then** et donnez-leur des noms lisibles. Un bon test se lit comme une spécification :

"Given an in-progress game, When the player guesses a word not in the dictionary, Then the attempt should be rejected"

Étape 4 — Gérer les cas limites

Ne vous limitez pas aux "Happy Paths". Les edge cases listés dans les règles (lettres multiples notamment) doivent être couverts par vos tests. Pensez aussi aux cas d'erreur et aux états invalides.

Étape 5 — Brancher une interface

Votre domaine doit pouvoir fonctionner **indépendamment** de tout système d'affichage. Le composant d'affichage (CLI, interface web, etc.) se base sur le modèle pour présenter l'information au joueur, mais **le modèle ne doit en aucun cas avoir conscience de l'affichage**.

C'est l'UI qui consomme votre domaine, pas l'inverse.

Isoler vos dépendances

Votre logique de jeu a besoin d'un **dictionnaire** pour valider les mots et en tirer un au hasard. Mais elle ne doit pas savoir *comment* ce dictionnaire est stocké (fichier, API, base de

données...).

Définissez une interface (un "contrat") pour cette dépendance et injectez-la dans votre jeu. Pour vos tests, utilisez une doublure (stub/mock/spy/fake) qui vous donne un contrôle total sur les mots disponibles et le mot secret choisi.

Livrables attendus

- Le lien vers votre dépôt Git (GitHub, GitLab, etc.).
- Le dépôt doit contenir un fichier `README.md` expliquant comment installer les dépendances, **lancer la suite de tests**, et **lancer l'application**.
- Une interface fonctionnelle (CLI ou frontend) permettant de jouer une vraie partie.

Évaluation

L'évaluation se fera en **revue de code** axée sur la qualité de votre modélisation et de votre suite de tests. Reportez-vous à la **Grille d'Évaluation** pour le détail des attentes.

Axe	Poids
Modélisation (types expressifs, domaine pur, séparation métier/infra)	30%
Qualité des tests (lisibilité, nommage, structure Given/When/Then, edge cases)	20%
Isolation (doublures, injection de dépendances, déterminisme des tests)	25%
Application fonctionnelle (CLI ou frontend opérationnel)	25%